

Iklwa Sentinel

Product Reference

Local-first network visibility and threat detection.
How it is built, how to run it, and how to control it.

Version	0.1.0
Database schema	v2
Source files tracked	155 files
Automated tests	see `cargo test`
Licence	Apache-2.0
Maintainer	IklwaLabs
Repositories	IklwaLabs/assegai-sentinel, ericalfonce/assegai-sentinel

This document is generated

Every figure, list and configuration field in this reference is read out of the repository by `tools/generate-reference-pdf.py`. Where the document and the code disagree, the document is wrong. Regenerate it after changing the code.

IklwaLabs

Contents

1	What Sentinel is	3
2	Architecture	3
3	The packet pipeline	5
4	Running it	7
5	Controlling it: the CLI	9
6	Controlling it: the desktop application	10
7	Configuration reference	12
8	Storage and retention	14
9	Platform support	15
10	Reading the interface	16
11	Operating limits	16
12	Security posture	17
13	Design decisions	18
14	Extending it	18
15	Troubleshooting	20

1 What Sentinel is

Sentinel watches the traffic on your machine, reconstructs the connections that carry it, stores what it finds in a local database, and shows you what is happening. There is no account, no cloud service, and no code path that opens an outbound connection.

It does

- Enumerate the network interfaces that can actually be captured, and say why any that cannot.
- Capture from a selected adapter on a dedicated thread, with backpressure that is counted rather than hidden.
- Decode Ethernet, VLAN, ARP, IPv4, IPv6, ICMP, TCP and UDP with its own bounds-checked decoder.
- Reconstruct bidirectional connections, identifying which side opened them.
- Aggregate traffic into one-second buckets, split by direction.
- Analyse saved PCAP files through the same pipeline used for live capture, so the two agree.
- Report packet loss from the driver, the queue and the decoder separately, on every surface.
- Store connections in one batched SQLite writer thread, with configurable retention.

It does not yet do

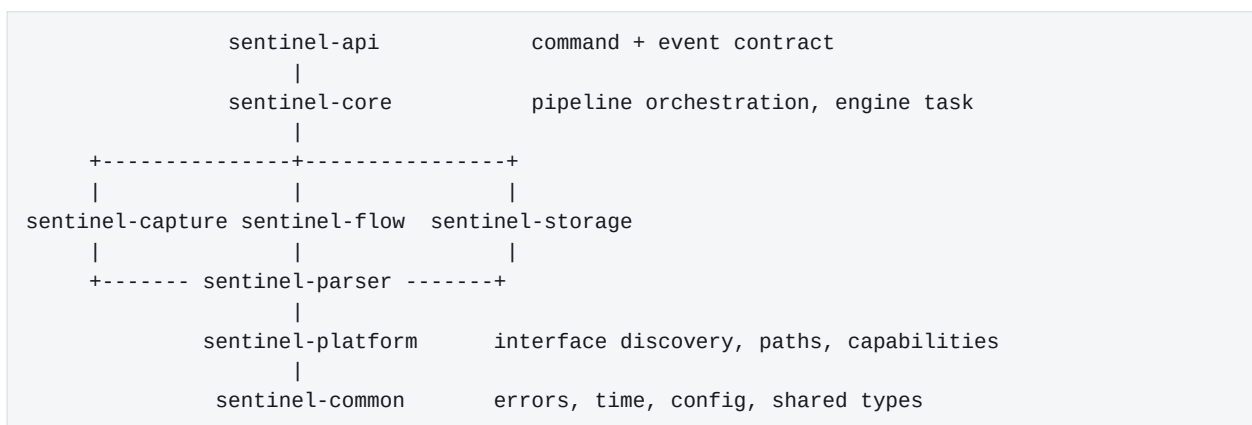
Threat rules, process attribution, device inventory, DNS correlation and desktop capture-file import are not implemented. They are absent from the interface rather than faked: there are no disabled buttons showing fabricated results, and a connection's risk score is shown as "not yet assessed" rather than as zero, because zero would read as safe, which Sentinel has not earned.

Reading this document

Sections 2 and 3 explain the design and are for anyone modifying Sentinel. Sections 4 to 6 are for running and controlling it. Sections 11 and 15 cover what to do when something is wrong or a number looks wrong.

2 Architecture

The workspace is nine crates plus two applications. Dependencies point strictly downward: a crate never imports a crate that depends on it. There is no shared "utils" bucket and no circular reference, because both would let two features grow entangled before anyone noticed.



Crates

Crate	Responsibility
<code>sentinel-api</code>	The public command and event contract for Iklwa Sentinel frontends
<code>sentinel-capture</code>	Packet capture adapters for Iklwa Sentinel: live capture, offline PCAP sources and the bounded ingress queue
<code>sentinel-common</code>	Shared types, errors, configuration and metrics for Iklwa Sentinel
<code>sentinel-core</code>	Engine orchestration for Iklwa Sentinel: the packet pipeline, engine sessions and the command/event boundary
<code>sentinel-flow</code>	Bidirectional flow keying, flow tables and traffic aggregation for Iklwa Sentinel
<code>sentinel-parser</code>	Packet decoding for Iklwa Sentinel: L2-L4 normalization plus a dependency-free PCAP file reader
<code>sentinel-platform</code>	Operating system integration for Iklwa Sentinel: interface discovery, capabilities, application paths and settings persistence
<code>sentinel-storage</code>	SQLite persistence for Iklwa Sentinel: schema migrations, batched writes and retention

The four load-bearing rules

- **The engine owns every decision.** The CLI and the desktop application are both thin clients over one engine, so they cannot disagree about what your network is doing. The frontend performs no analysis.
- **One owner of mutable state.** A single engine task owns the flow table, so the packet path needs no locks and the ownership of that state is obvious from reading one file.
- **Platform code lives in one crate.** Only `sentinel-platform` contains `cfg(target_os)`. Everything else consumes normalized types, which is why the same pipeline runs on every platform.
- **Capture is not analysis.** `libpcap` blocks, so capture owns a dedicated OS thread and publishes into a bounded queue. Falling behind shows up as a counted drop, not as a stalled interface.

Why the queue is bounded

An unbounded queue turns a slow disk into an out-of-memory kill. Blocking the reader pushes the loss into the kernel, where it is harder to attribute. Dropping the frame and incrementing a counter is the only option that keeps the loss visible, and visibility is the point: a total derived from lossy data, presented as complete, is a correctness bug.

3 The packet pipeline

Every frame takes the same path whether it came from a live adapter or a PCAP file. That is the single most important property of the design, and it is what makes offline analysis trustworthy: it is not a second implementation that happens to agree, it is the same code.

```
wire
|
[1] capture thread      libpcap blocks; copies bytes into a bounded queue
|
v   bounded queue, drop-counted
[2] engine drain        every 25ms, drain all available, bounded per tick
|
[3] decode              Ethernet > VLAN > IPv4/IPv6 > TCP/UDP/ICMP
|                        failures counted, never fatal
[4] flow key            canonically ordered endpoint pair + protocol
[5] flow table          lookup or create; update; mark direction
[6] aggregation         one-second buckets, upload/download split
|
+---> [7] snapshot      every 250ms, throttled, to the interface
|
[8] storage             every 5s, batched, in the writer thread
```

Timings and bounds

Setting	Value
Drain interval	25
Snapshot interval	250
Flush interval	5
Maintenance interval	30
Chunk size	4,096
Frames per tick	250,000

Where data can be lost, and how you find out

Loss	Counted as	How it surfaces
Driver could not keep up	CaptureStats::dropped	Frames lost, and a notice in the overview
Driver reported interface drops	CaptureStats::interface_dropped	Same counters, reported separately
Engine queue full	ingress drop counter	stats.queue.dropped
Frame truncated by snaplen	truncated_packets	Decoder counters
Frame undecodable	decode_errors	Decoder counters; analysis continues

Decoder decisions worth knowing

- **Non-initial IPv4 fragments never become flows.** They have an IP header but no ports. Keying one would create a second, portless flow for a connection that already has one, splitting its byte counts in half. They are counted and excluded.

- **Truncation is not an error.** A small snaplen produces short frames. That is reported as truncated, separately from malformed, because the two mean different things.
- **ARP is decoded but has no flow.** It is real traffic with no connection identity.
- **ICMP is a portless flow.** It is tracked rather than dropped, with no port to attribute.
- **Direction and locality are separate facts.** The flow key is canonically ordered, so source/destination comes from initiator detection, while local/remote is filled only when an endpoint matches this machine. Showing the canonical order in a column labelled "remote" would display your own address to you.

4 Running it

Prerequisites

Requirement	Version	Why
Rust	1.95 or newer	Edition 2024, plus AtomicUsize::try_update
Node.js	20 or newer	Desktop frontend build only
npm	10 or newer	Ships with Node 20
C toolchain	current	bundled SQLite and libpcap bindings
Npcap (Windows)	SDK or runtime	capture driver; see below
libpcap-dev (Linux)	any	capture driver
BPF (macOS)	built in	no install required

The Windows linking problem

Npcap's installer puts `wpcap.dll` in `System32` but not `wpcap.lib`. A machine with the runtime but not the SDK can capture yet cannot link, and the error is a bare linker message about a missing file. Three ways out, in order of preference:

- Install the Npcap SDK. The build script finds it automatically. This is what CI does.
- Set `IKLWA_PCAP_LIB` to a directory containing `wpcap.lib`. Use this on CI images.
- Run `tools/make-wpcap-lib.ps1` to synthesise an import library from the installed DLL's export table. A local build aid for machines that cannot install the SDK; not something to ship.

If no import library is found the build still succeeds and emits a warning explaining what to do. That is deliberate: an actionable warning beats a link error containing a path.

Build and run

```
# 1. Check whether this machine can capture at all
cargo run -p sentinel-cli -- doctor

# 2. List what is available
cargo run -p sentinel-cli -- interfaces

# 3. Analyse a capture file. No privileges needed, so this always works
cargo run -p sentinel-fixtures -- fixtures/demo-traffic.pcap
cargo run -p sentinel-cli -- analyze fixtures/demo-traffic.pcap

# 4. Monitor for 30 seconds. Needs an elevated shell
cargo run -p sentinel-cli -- capture --duration 30

# 5. Desktop application
cd apps/desktop/frontend
npm install
cd ../../
cargo build --release -p sentinel-desktop --features custom-protocol
./target/release/sentinel-desktop
```

The custom-protocol feature is not optional

Building the desktop binary without it produces a window that is simply black: no error, no log line. The build loads the Vite dev-server URL instead of the assets compiled into the executable, and with no dev server running that page never resolves. Use `tauri dev` for development and pass `--features custom-protocol` for anything you intend to run or distribute.

Verifying an installation

- `sentinel doctor` reports the backend, whether the driver is present, the process privilege state, and whether capture is possible right now.
- `sentinel version` prints the build version and the database schema it expects (v2).
- `sentinel analyze fixtures/demo-traffic.pcap` should report 14 packets and 8 connections. If it does not, the decoder or flow engine is broken, not your network.

5 Controlling it: the CLI

Every subcommand is a client over the same engine the desktop application uses. Data goes to stdout and messages to stderr, so `--json` output pipes cleanly into another tool.

Command	What it does	Options
<code>sentinel interfaces</code>	List every network interface, with addresses and whether it can be captured	—
<code>sentinel capture</code>	Monitor an interface live and print statistics as they change	<code>--interface</code> <code>--duration</code> <code>--connections</code>
<code>sentinel status</code>	Report the current session state, totals and loss counters	—
<code>sentinel analyze</code>	Analyse a PCAP file and report the connections it contains	<code>--all</code>
<code>sentinel export</code>	Write stored connections to CSV or JSON	<code>path</code> <code>--json</code> <code>--limit</code>
<code>sentinel doctor</code>	Report whether capture is possible here, and what to do if not	—
<code>sentinel prune</code>	Delete stored data older than the retention period	<code>--dry-run</code>
<code>sentinel version</code>	Print the build version and the database schema it expects	—

Global flags

Flag	Effect
<code>-v, -vv</code>	Raise log verbosity: warnings, then info, then debug
<code>-q, --quiet</code>	Suppress informational notes on stderr
<code>--json</code>	Emit machine-readable JSON instead of formatted text

Worked examples

```
# Is capture possible here, and if not, why not?
sentinel doctor

# Thirty seconds of live monitoring on a specific adapter, one line per connection
sentinel capture --interface 12 --duration 30 --connections

# What is in this capture file?
sentinel analyze session.pcap
sentinel analyze session.pcap --all          # every connection, not just the top 20

# Export stored history
sentinel export --limit 500                  # CSV to the exports directory
sentinel export report.json --json          # JSON to a path you choose

# What would retention delete?
sentinel prune --dry-run
sentinel prune
```

Note that `sentinel analyze` requires no privileges and no driver, because it reads a file. It is the fastest way to confirm Sentinel works on a machine that cannot capture.

6 Controlling it: the desktop application

First run

- The interface picker opens automatically, listing every adapter with its addresses and whether capture is possible.
- An adapter Sentinel cannot open is listed with the reason, not hidden. "Why is my Wi-Fi missing" is the most common first-run question, and an absent entry cannot answer it.
- Selecting an adapter and pressing **Start monitoring** begins capture.
- Press **Stop monitoring** to finish. The session is flushed to storage on stop.

Views

View	What it shows
Overview	Live rates, totals, the traffic chart, the busiest connections, protocol mix, and any data-loss notice
Traffic	The traffic chart at full size
Connections	The sortable, filterable connection table with a detail panel
PCAP analysis	Analyse a capture file through the same engine as live monitoring
Interfaces	Adapter inventory with capture readiness and reasons
Settings	Privacy, capture, retention and diagnostics, plus where data is stored

Controls that change behaviour

- **Start / Stop monitoring** in the top bar. The status dot and label beside it always reflect the engine's real state, not a local guess.
- **Change interface** switches adapters, and resets per-session counters, because a new adapter's traffic is not comparable with the previous one's.
- **Pause** in Connections freezes the list while leaving capture running, so you can read a row without it moving under the cursor.
- **Settings** changes take effect immediately and are written to disk. An out-of-range value is rejected at the boundary with an explanation, before it can break the next start-up.

The interface contract

The window talks to the engine through nine commands and receives one event stream. Commands are requests; the event stream carries aggregate snapshots and state changes. There is no per-packet event, because a UI that receives 10 000 messages a second cannot stay responsive.

Command	Purpose
list_interfaces	Adapters available for monitoring, with addresses and readiness
capabilities	What this process can do: driver, privilege state, capture limits
start_monitoring	Begin capture on the chosen adapter

Command	Purpose
stop_monitoring	End the session and flush pending writes
analyze_capture	Analyse a PCAP file through the same engine
get_snapshot	Fetch the full state for an immediate render
get_settings	Read the effective configuration
save_settings	Replace and persist the configuration
get_diagnostics	Data locations and any start-up warnings

Tauri 2 capabilities

Tauri 2 denies every permission not explicitly granted, so `capabilities/default.json` is required. If it is missing, the window loads but every command fails with an error the interface cannot classify. If you see that symptom with a working window, add the file rather than debugging the engine.

7 Configuration reference

Configuration lives in one JSON file under your user data directory, and the running engine is always the authority: the file is read at start-up, and Settings shows the effective values rather than the file's contents. Unknown keys are rejected, so a typo is loud rather than silently ignored.

```
Windows %LOCALAPPDATA%\IklwaLabs\Iklwa Sentinel
macOS   ~/Library/Application Support/Iklwa Sentinel
Linux   ~/.local/share/iklwa-sentinel
```

Capture

Field	Type	What it does
interface_id	Option<String>	The adapter to monitor. Written when you pick one from the picker.
snaplen	u32	Bytes captured per frame. 65535 keeps whole frames including jumbo frames.
promiscuous	bool	Also capture traffic not addressed to this machine. Sees more, unrelated traffic.
read_timeout_ms	i32	How long a read waits before checking for stop requests.
queue_capacity	usize	Frames buffered between capture and analysis. Beyond this, frames are dropped and counted.
flow_idle_timeout_secs	u64	How long a quiet connection stays in the live table before its totals are kept but it leaves.
max_tracked_flows	usize	Cap on simultaneously tracked connections. Oldest is evicted when full.
ui_update_hz	u32	How often the interface refreshes. The main CPU lever on a busy network.

Retention

Field	Type	What it does
flows	RetentionPeriod	How long stored connections are kept.
traffic_samples	RetentionPeriod	How long the traffic chart's samples are kept.

Privacy

Field	Type	What it does
capture_payload_samples	bool	Retain small payload excerpts as evidence. Off by default; Sentinel is built to work without retaining payload bytes.
redact_local_addresses_in_exports	bool	Replace this machine's addresses in exported files.

Appearance

Field	Type	What it does
theme	ThemeMode	Dark or follow the system.
compact_tables	bool	Tighter row height in the connections table.

Advanced

Field	Type	What it does
log_level	LogLevel	Applied at start-up. Verbose is useful when reporting a problem.
wal_mode	bool	Write-ahead logging. Improves durability and concurrent read performance.
write_batch_size	u32	Rows batched into one transaction before the writer flushes.
max_db_size_mb	u32	Soft ceiling for the database file; retention is what actually bounds it.

Settings that matter most

Setting	Guidance
uiUpdateHz	How often the interface refreshes. Lower it on a busy network; it is the main CPU lever.
promiscuous	Captures traffic not addressed to you. Needs elevation, and sees far more unrelated traffic.
snaplen	Bytes per frame. 65535 keeps whole frames; lower values are faster but truncate.
flows_retention	Shorter keeps the database small. Sentinel never stores raw packets, so this is metadata only.
capturePayloadSamples	Off by default. Turning it on stores small payload excerpts as evidence for future rules.

8 Storage and retention

Everything is stored in one SQLite file, written by a single dedicated thread. One writer, one connection: SQLite has a single write lock, so a second connection would contend for it and the loser would report a spurious "database is locked" to you. Retention runs inside that writer's transaction for the same reason.

Schema

Table	Holds
settings	Key/value application settings
interfaces	Adapters seen, so the inventory is useful without rediscovery
flows	Reconstructed connections with per-direction byte and packet counts
traffic_samples	One-second upload/download buckets for the chart
protocol_totals	Cumulative per-protocol totals, written as session deltas

Migrations are plain SQL files, embedded at compile time, appended in order. Current schema version is **2**, across 2 migrations (0001_initial.sql, 0002_protocol_totals.sql). Each runs in its own transaction, so a failure leaves the database at the last good version rather than half-migrated.

Retention

- Connections and traffic samples are deleted once they are older than the configured period.
- The sweep runs every 30 seconds while the engine is running, and inside the writer's transaction.
- `sentinel prune --dry-run` shows what would be removed without removing it.
- Sentinel stores connection metadata, not packets. There is no column anywhere in the schema for payload bytes.

Flush behaviour

Connections are written once they close or go quiet for ten seconds. Active ones stay in memory for the live view and are written later, so killing the application does not lose a long-lived connection. Protocol totals are written as a delta since the last flush, never as the session total, which would double-count on every flush.

9 Platform support

	Windows	Linux	macOS	Android
Driver	Npcap	libpcap	BPF (built in)	Tun2Socks (planned)
Privilege	Administrator	CAP_NET_RAW or root	access_bpcap group	app-private VPN
Live capture	Yes	Yes	Yes	Planned
Interface list	Yes	Yes	Yes	Yes
PCAP analysis	Yes	Yes	Yes	Yes
Loopback	Dedicated adapter	Yes	Limited	n/a
Desktop app	Yes	Yes	Yes	No

Per-platform notes

- **Windows.** Npcap in place of or alongside WinPcap. Loopback traffic appears on a separate adapter, available only in Npcap's full install mode; Sentinel says which mode is required rather than showing an adapter that captures nothing.
- **Linux.** Prefer granting the binary capability over running as root: `sudo setcap cap_net_raw,cap_net_admin=eip ./sentinel`.
- **macOS.** No driver to install. The process must be in the `access_bpcap` group, which may need an administrator.
- **Android.** Interface enumeration and PCAP analysis work; live capture is the main outstanding platform work and needs a userspace stack over a VPN slot.

A platform counts as supported when Sentinel can enumerate interfaces, capture from a selected one, analyse a file, and store results, without special-casing the pipeline. An interface listed as capturable that captures nothing is treated as a bug, not a platform limitation.

10 Reading the interface

A few conventions carry across every view, so that a number means the same thing wherever it appears.

Convention	What it means
Source / Destination	Which side opened the connection, inferred from port roles. The detail panel says when the direction could not be determined.
Local / Remote	Filled only when an endpoint matches one of your interfaces. Blank means the traffic does not involve this machine, not that it is missing.
Upload / Download	Relative to this machine: what you sent, and what you received.
Frames lost	Packets the driver or the queue dropped. Non-zero means every total shown is incomplete.
Not yet assessed	A risk score that has not been computed. Never rendered as zero, because zero reads as safe.
Unavailable (interface)	Sentinel cannot capture from this adapter, and the row states why.

Severity is never the only signal

Colour is decorative; the text carries the meaning. Every status is a word as well as a colour, because a colour-only signal is unreadable to a meaningful fraction of users and useless in a screenshot printed in black and white.

11 Operating limits

Know these before you rely on a number. Each is a deliberate bound rather than an accident, and each is visible at runtime.

Bound	Value	Consequence
Idle flow timeout	120 s	A quiet connection leaves the live table but keeps its totals
Maximum tracked flows	65 536	Oldest is evicted when full; creation counts stay accurate
Snapshot connections	200	The interface shows the most recent 200, not all of them
Traffic series	one-second ring	The chart shows recent history, not the whole session
Queue capacity	32 768 frames	Beyond this, frames are dropped and counted
Flush interval	5 s	Up to five seconds of connections may not yet be on disk

On a very busy link

If the loss counters are non-zero and rising, the machine cannot keep up. Lower `uiUpdateHz`, reduce `snaplen`, or disable promiscuous mode. If drops persist at low traffic, the disk or the driver is the bottleneck, and the counters distinguish which: a high driver-drop count with a low queue watermark means the driver.

12 Security posture

What Sentinel does with your data

This is a property of the code rather than a promise, and you can check it:

```
# No HTTP, TLS or websocket client anywhere in the engine's dependency graph.
cargo tree -p sentinel-core | grep -Ei "reqwest|hyper|curl|rustls|tokio-tungstenite"
# Expects no output.

# No code path builds an outbound connection.
grep -rn "TcpStream|UdpSocket|connect(" crates/ apps/ --include=*.rs
# Expects no matches outside tests and the capture driver bindings.
```

Property	Status
Outbound network connections	None. No HTTP client, no resolver, no telemetry, no update check.
Packet payloads stored	No. Headers, flows and aggregates only; the schema has no payload column.
Traffic modification	None. Sentinel reads. No injection, spoofing or shaping.
Account or sign-in	None.
Required privileges	Elevated, for reading frames from an adapter. Nothing more.
Filesystem reach	Its own data directory only.
Window content policy	Scripts and styles from the bundle only; no remote code.

Threat model boundaries

- An attacker with administrator rights on your machine can read anything Sentinel can, and can alter its binary. Sentinel does not defend against that.
- Promiscuous mode sees traffic that was never meant for you. That is its purpose, and it is why it is off by default.
- The local database is not encrypted at rest. Protecting it is the operating system's job, through filesystem permissions.
- Detection is deliberately offline. A user in an air-gapped environment gets the same analysis as everyone else, and an adversary cannot poison verdicts by feeding the detector.

13 Design decisions

The choices worth arguing about, with the condition that would reverse each one. A decision with no stated reversal condition is a decision nobody has thought about.

#	Decision	Why	Revisit when
D1	`rusqlite` with bundled SQLite over `sqlx`	Synchronous, single-writer batching is simpler to reason about than an async pool for a single local database; fewer build-time dependencies	A remote/multi-writer store is ever needed
D2	`pcap` crate for Windows/Linux/macOS	One well-maintained binding over Npcap/libpcap/BPF; identical API surface reduces platform code	Android TUN requires a different adapter (already the case)
D3	Flow keys normalized bidirectionally at construction	Connections are inherently bidirectional; ordering in the key makes aggregation correct without per-packet state	Never
D4	Flow state owned by one engine task	Avoids lock contention in the hot path and keeps ownership obvious	Multi-threaded decoding is required for >100k pps
D5	Migrations embedded as SQL files, not a runtime framework	Reproducible builds, reviewable diffs, no extra dependency	Never
D6	Config is plain typed data with `serde` defaults; persistence lives in the platform layer	Keeps `sentinel-common` free of OS dependencies and makes config testable	Never
D7	No telemetry, no crash upload, no account	Product requirement and the trust differentiator	Cloud sync ships as opt-in
D8	Frontend owns no security logic	Keeps analysis testable in Rust and portable to CLI/sensor	Never

Deliberately not done

- No global mutable singleton or god object holding every subsystem.
- No separate event-bus crate: the engine broadcaster already owns event dispatch.
- No empty crate as a placeholder. Additional crates are introduced by the milestone that needs them, because a placeholder is a promise the product has not kept.
- No third-party decoding crate. Sentinel owns its L2-L4 decoder because the flow engine depends on seeing truncation and fragmentation explicitly.

14 Extending it

Adding a threat rule

- A rule takes a flow plus its context and returns a finding with a severity and a one-sentence explanation. Most interesting signals are not in a single flow: a beacon is a timing regularity, a scan is a set of destinations.
- Rules run inside the engine on the task that owns the flow table, after aggregation, throttled per flow, and deterministically. A capture file replayed twice must produce identical findings.
- Every finding must explain itself in one sentence. A score with no explanation either gets ignored or over-trusted, and both are bad.

- Each rule needs a positive case, a negative case, a boundary case, and a benign pattern that resembles it. The last one decides whether the rule is usable.

Adding a capture backend

- Implement `CaptureProvider`, keep every `cfg(target_os)` inside `sentinel-capture`, and map failures onto the existing error variants so the user-facing message stays built in one place.
- Report driver-level drops through `CaptureStats`. A silent loss is worse than no capture.
- Write tests that do not need the driver: queue behaviour, error classification and config translation are all testable without hardware.

Changing the engine contract

Any change to a serializable type must change the hand-written TypeScript mirror in `packages/types` in the same commit, including the field-name casing. They are not generated, so the mirroring is a deliberate act rather than an automatic one.

Before opening a change

```
cargo fmt --all
cargo clippy --workspace --all-targets -- -D warnings
cargo test
cd apps/desktop/frontend && npm run typecheck && npm run lint && npm run build
```

15 Troubleshooting

Symptom	Likely cause	Do this
Black window, title bar only	Built without custom-protocol, so it loads the dev-server URL	Rebuild with <code>--features custom-protocol</code> . See section 4.
Window loads, every command fails	capabilities/default.json missing or incomplete	Add the file. Tauri 2 denies anything not explicitly granted.
LNK1181: cannot open wpcap.lib	Npcap runtime installed but not the SDK	Install the SDK, or set <code>IKLWA_PCAP_LIB</code> . See section 4.
Capture permission denied	Process is not elevated	Run as administrator, or grant <code>CAP_NET_RAW</code> on Linux.
Interface listed but not available	Adapter is down, or the driver refuses it	The row states the reason. Bring the adapter up, or choose another.
Frames lost is non-zero	Machine cannot keep up with the packet rate	See section 11. Compare driver drops against queue watermark to find which.
Database is locked	Two Sentinel processes are running	Close the other one. SQLite allows one writer.
Live and offline results differ	This is a bug	They are the same pipeline. Please report it with a capture file.

Getting help

- Include `sentinel doctor` output; it names the driver, the privilege state and the data directory in one go.
- Attach the log directory's contents, shown under Settings. Logs go there, not to a console, because a GUI application has no console on Windows.
- If the problem involves a capture file, attach it. You can inspect it yourself first with `sentinel analyze`.
- Security issues go to security@iklwalabs.com, not the issue tracker.

Appendix Verification state

Every milestone is complete only when it is formatted, linted, tested and documented. This is the state at the time of printing.

Test counts were not available when this document was generated. Run `cargo test --workspace` for the current figures.

Generated from commit 3a9f263 on 2026-10-06 by `tools/generate-reference-pdf.py`. Regenerate rather than editing this file by hand.